D4: Architecture and Engineering Baseline

J&J Hub Boutique Management System

Item	Detail
Team	The Company (team repository seg26-48-j-j-hub)
Client	Mrs. Juliet Amponsah, physical dress boutique (front-row showroom floor plus detached backroom storage)
Course	DCIT 208 Software Engineering, 2026 session
Version / release tag	v0.0-baseline
Prepared by	[Team member names and roles: add before submission]
Repository / board	[Team repo URL] / [Project board URL]

Scope. This document is the team's architecture and engineering baseline. It states what the system is, how its parts fit together, which decisions we made and why, and how the repository protects quality. It builds on the team's discovery work and on ADR-001 (database choice), which was developed in individual assignment IA4. Items marked *draft* are proposals the team should confirm before submitting.

A. Architecture report

A1. System context

The system serves three kinds of people and depends on three external services. The key business problem is that stock is tracked in two physical places and sold through two channels (walk-in and online), so one authoritative store of stock is required.

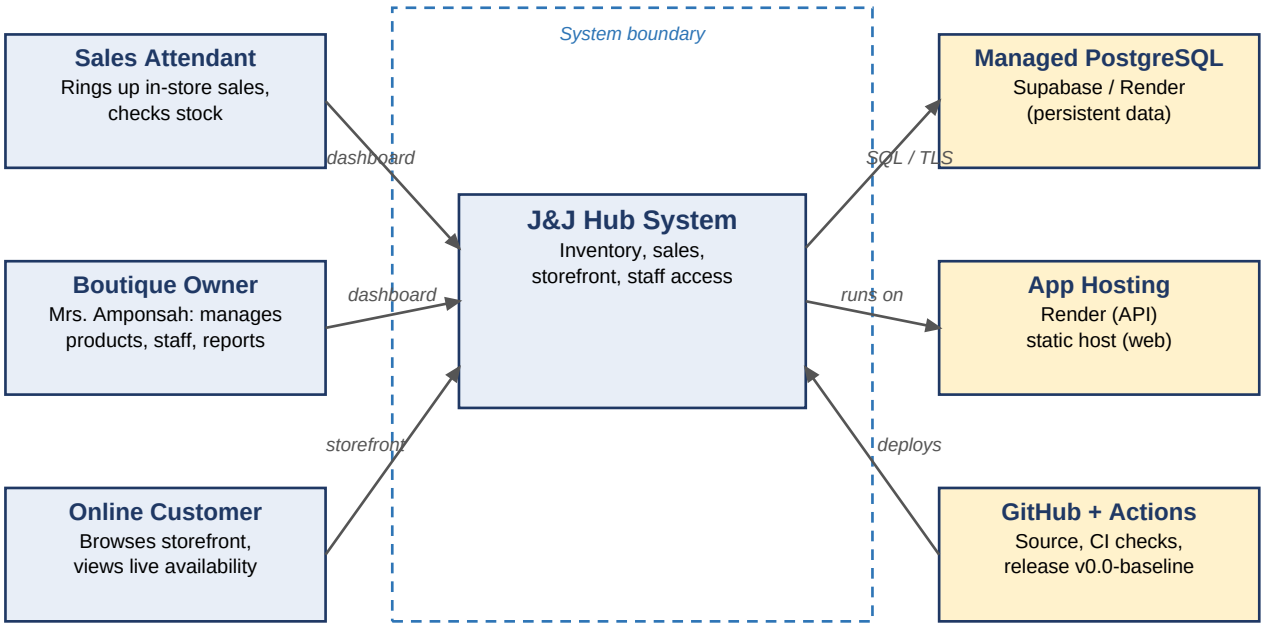


Figure 1: System context diagram showing users, external systems and the system boundary.

Actor / system	Role and boundary
Sales attendant	Uses shop tablet or desktop on stable broadband; creates in-store orders, checks stock, moves stock between floor and backroom.
Boutique owner	Manages products and variants, staff accounts and sales reports.
Online customer	Anonymous visitor; reads the storefront only. Cannot see staff or financial data.

Actor / system	Role and boundary
Managed PostgreSQL	External, free academic tier (Supabase or Render). Only the API connects to it.
App hosting	Runs the API and serves the web client over HTTPS.
GitHub and Actions	Source control, pull-request checks and the release tag.

A2. Container and component view

J&J Hub is a three-tier web application. The presentation tier is a React and TypeScript client (a staff dashboard and a public storefront). The application tier is a Node.js and Express REST API. The persistence tier is a PostgreSQL database. All business rules, including stock checks, run in the API and database, never in the browser.

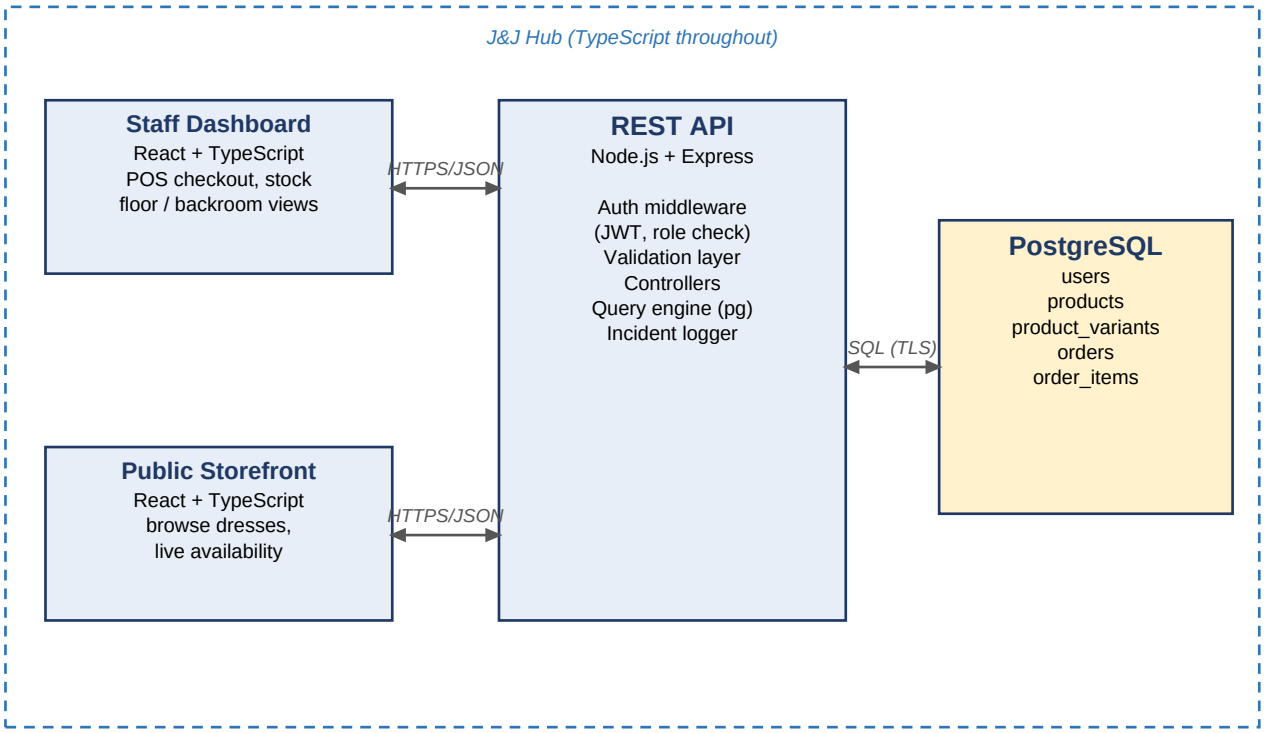
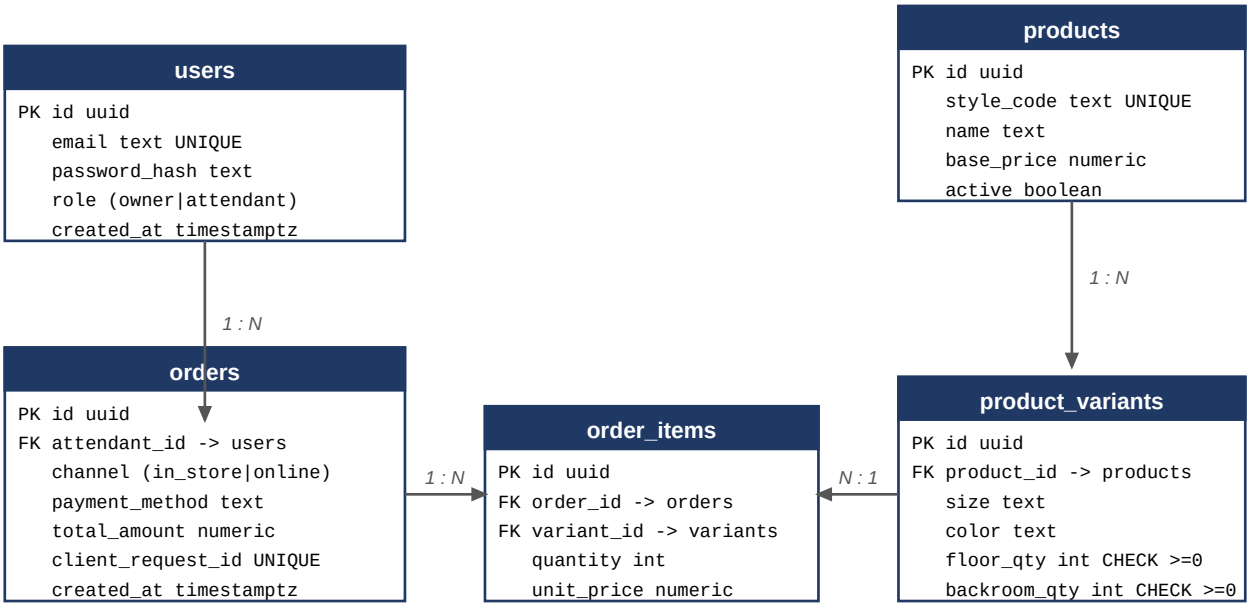


Figure 2: Container and component diagram (draft: confirm hosting split with the team).

Component	Responsibility
Staff dashboard	POS checkout form, stock views for floor and backroom, product management. Keeps a pending checkout in local state so it survives a failed request.
Public storefront	Read-only product browsing with live availability.
Auth middleware	Verifies the JWT and enforces role (owner or attendant) before any controller runs.
Validation layer	Rejects malformed requests (types, ranges, required fields) before they reach the database.
Controllers	One per resource (auth, products, variants, orders). Orchestrate use cases.
Query engine	Parameterised SQL through the pg driver. Wraps checkout in a single transaction.
Incident logger	Writes high-priority entries for failed requests (see failure scenario in section A6).

A3. Data model

The model follows ADR-001: a parent product (style) links to child variants (size and colour), and each variant carries both floor and backroom quantities. Foreign keys prevent orphan rows, and CHECK constraints prevent negative stock, which is the database-level guard against overselling.



A variant has one row per size/colour. Floor and backroom stock sit on the same row, so every sale and transfer is one atomic update.

Figure 3: Entity relationship diagram (draft schema).

Schema draft (core of the baseline migration):

```
CREATE TABLE products (id uuid PRIMARY KEY DEFAULT gen_random_uuid(), style_code text UNIQUE NOT NULL, name text NOT NULL, base_price numeric(10,2) NOT NULL, active boolean DEFAULT true);
CREATE TABLE product_variants (id uuid PRIMARY KEY DEFAULT gen_random_uuid(), product_id uuid NOT NULL REFERENCES products(id), size text NOT NULL, color text NOT NULL, floor_qty int NOT NULL CHECK (floor_qty >= 0), backroom_qty int NOT NULL CHECK (backroom_qty >= 0), UNIQUE (product_id, size, color));
```

A4. API and interface contract

REST over HTTPS with JSON bodies. Staff routes need a bearer token; the storefront route is public and read-only.

Method and path	Access	Request / response summary
POST /api/auth/login	Public	Body: email, password. 200: token, role. 401 on bad credentials.
GET /api/storefront/products	Public	200: list of active products with variants and total available quantity (no cost or staff data).
GET /api/products	Staff	200: products with variants and floor_qty / backroom_qty.
POST /api/products	Owner	Body: style_code, name, base_price, variants[]. 201: created product. 409 if style_code exists.
PATCH /api/variants/:id/stock	Staff	Body: floor_delta, backroom_delta (transfer or adjustment). 200: new quantities. 422 if either would go below 0.

Method and path	Access	Request / response summary
POST /api/orders	Staff	Body: client_request_id, channel, payment_method, items[{variant_id, quantity}]. 201: order_id, total, items. 409 insufficient stock. 503 database unavailable.
GET /api/orders	Owner	Query: from, to. 200: orders with items for reporting.

Idempotency. POST /api/orders requires a client_request_id (UUID). The database enforces uniqueness, so a retry after a failure cannot create a duplicate sale. This is what makes the 30-second retry in the failure walkthrough safe.

A5. Architecture Decision Records

Four decisions are recorded. ADR-001 was completed and submitted in IA4. ADR-002 to ADR-004 are drafts written in the same format for the team to confirm.

Field	ADR-001: PostgreSQL for core transactional data (Accepted)
Context	Stock is split between showroom floor and backroom and sold in-store and online. Stock counts must update immediately and never oversell. About 500 SKUs and concurrent weekend checkouts.
Options	A: PostgreSQL relational database (chosen). B: MongoDB document database (rejected).
Decision	PostgreSQL. It gives strict parent-child relations, ACID transactions and row locking out of the box, and fits the team's existing relational skills.
Evidence	Weighted matrix (IA4): client/data fit, security/privacy and maintainability rated High and favour PostgreSQL; team skill Medium favours PostgreSQL; cost is equal (both have free tiers); MongoDB only wins on massive-log scale, which J&J Hub does not need.
Consequences	Benefits: atomic checkout, no orphan rows, Supabase integration. Costs: schema changes need migrations. Risk: lock contention under extreme spikes.
Revisit when	The client moves to a multi-tenant marketplace with unstructured item attributes.
Verification	Sprint 1 walking skeleton: local PostgreSQL container and a script that fires concurrent purchases at one variant row to confirm locking and accurate counts.

Field	ADR-002: TypeScript with React and Express (Draft)
Context	The team must deliver within one semester using free tooling, and already uses TypeScript for the schema and API work.
Options	A: React + Express, TypeScript on both sides (chosen). B: Server-rendered framework such as Django or Laravel. C: Next.js full-stack.
Decision	React client and Express API, both in TypeScript, sharing request and response types.
Rationale	One language lowers training cost, shared types catch contract mismatches at compile time, and a separate API keeps the dashboard and storefront on one backend.
Trade-offs	Two deployables to configure; more boilerplate than a full-stack framework.
Revisit when	Search-engine visibility of the storefront becomes a client priority (consider server rendering).

Field	ADR-003: JWT authentication with role-based access (Draft)
Context	Staff access must be limited by role, and staff credentials are sensitive.
Options	A: Email and password, bcrypt hashes, short-lived JWT, owner/attendant roles (chosen). B: Server sessions in the database. C: Third-party identity provider.
Decision	Option A, with role checks in the auth middleware and password hashes only in the users table.
Rationale	Stateless API that is simple to test and fits free hosting; the role model is small (two roles).
Trade-offs	Tokens cannot be revoked instantly; mitigated by short expiry and a secret rotation procedure.
Revisit when	More roles, staff self-registration or single sign-on are required.

Field	ADR-004: Managed free-tier cloud deployment (Draft)
Context	No budget; the client needs a reachable system, and a new developer must be able to reproduce it.
Options	A: Managed PostgreSQL on Supabase or Render, API on Render, web client on a static host (chosen). B: Self-hosted VPS. C: Run only on the boutique's local machine.
Decision	Option A, with a staging environment and a final-release environment (section C).
Rationale	Free academic tiers, automatic TLS, backups on the database host, and deploy-from-GitHub.
Trade-offs	Free tiers can sleep or throttle; the database is a single point of failure, handled by the recovery behaviour in A6.
Revisit when	The client needs guaranteed uptime or the boutique's traffic outgrows free limits.

A6. Failure scenario and resilience (from IA4)

The most important failure is a database connection loss during checkout. The table summarises the walkthrough verified in IA4.

Step	Behaviour
Normal path	Attendant presses complete transaction. Client sends POST /api/orders. The API runs one database transaction: insert order and items, reduce floor_qty, commit. Invoice is shown.
Failure	The database connection drops mid-request (cloud outage or network drop) before commit.
Detection	The Express API catches a connection timeout from the database driver and raises an explicit error.
User sees	A red banner: Transaction failed. Database server offline. Please retry in a few moments. The spinner stops.
Data at risk	None. The uncommitted transaction rolls back, so stock never drops without a matching order record.
Logging	High-priority incident log: timestamp, attendant user id, target SKU code and the raw timeout error. No passwords or tokens are logged.
Recovery	The client keeps the checkout in local state, disables the submit button and starts a 30-second retry counter. The retry reuses the same client_request_id, so no duplicate order is possible.
Test	Integration script that stops the database service mid-request and asserts that the UI alerts the cashier and that no partial rows exist.

A7. Security and privacy design assumptions

- **Credentials:** staff passwords are stored only as bcrypt hashes; plain passwords are never logged or stored.
- **Secrets:** database URL, JWT secret and keys live in environment variables, never in the repository; a .env.example file lists names only.
- **Transport:** all traffic uses HTTPS; the API-to-database link uses TLS.
- **Access control:** least privilege through two roles; the public storefront route exposes only product and availability data.
- **Injection and input:** parameterised queries only, plus request validation before database access.
- **Data minimisation:** no card data is stored. Payments are cash at launch; any mobile-money integration will use the provider's hosted flow.
- **Customer data:** online customers browse anonymously, so no personal data is collected in the baseline. Any future collection must follow Ghana's Data Protection Act, 2012 (Act 843).
- **Auditability:** every order records the attendant and timestamp; incident logs support investigation.
- **Assumptions to confirm:** staff use shop-owned devices on stable broadband; one boutique only (single tenant); about 500 SKUs.

B. Repository engineering baseline

The repository enforces quality from the first commit. The table maps each required control to its file and the rule it applies. Status is updated at submission.

Control	File	What it does
Root Makefile	Makefile	Targets: setup (npm ci for client and server), lint (ESLint), test (Jest and Vitest), run (start API and client), build.
CI workflow	.github/workflows/ci.yml	On pull requests to main: checkout, Node setup, make setup, make lint, make test. Fails the PR on any error.
PR template	.github/pull_request_template.md	Requires linked issue, tests run, screenshots for UI changes and an AI-assistance disclosure.
Issue templates	.github/ISSUE_TEMPLATE/	Three forms: feature, bug, documentation task.
CODEOWNERS	.github/CODEOWNERS	Review required for Makefile, workflows, package files and docs/04_architecture_ci.

Control	File	What it does
Branch protection	GitHub settings	No direct commits to main; pull request with approving review and passing CI required.

C. Deployment and environment plan

- **Platform:** API on Render, PostgreSQL on Supabase (or Render Postgres), web client on a free static host. Chosen for zero cost and deploy-from-GitHub (ADR-004).
- **Environment variables:** DATABASE_URL, JWT_SECRET, JWT_EXPIRES_IN, CORS_ORIGIN, NODE_ENV. Real values are set in the host dashboards and a local .env that is git-ignored; the repository holds only .env.example.
- **Local setup (new developer):** clone the repo; copy .env.example to .env; start a local PostgreSQL container; run make setup, then the migration, then make run.
- **Staging and release:** merges to main deploy to staging. After the team verifies checkout and stock flows there, the tagged release is promoted to the final environment.
- **Backup and rollback:** rely on the database host's automatic backups plus a manual dump before each release; roll back by redeploying the previous tag and restoring the pre-release dump if a migration was applied.

E. Baseline release (v0.0-baseline)

The release notes must separate what exists from what is only planned. Use this structure when creating the GitHub release:

Section	Content to state in the release notes
Exists	Architecture report, repository structure, Makefile, CI workflow, templates, CODEOWNERS, mid-sem defence deck.
Scaffolded	Client and API project skeletons, database schema draft and migration file.
Tests that run	[Fill in: for example the lint job and the placeholder-free unit test that CI executes]
Not yet implemented	Checkout flow, stock transfer, storefront, authentication endpoints, concurrency test.
CI status	Passing, or each failure explained with an instructor-approved remediation plan.

Verification plan

In Sprint 1 the team will build a walking skeleton: a local PostgreSQL test container plus a component test that simulates concurrent purchases on a single variant row. It confirms that row locking works and that counts stay correct under load before any production deployment.